

**ÇANKAYA UNIVERSITY  
COMPUTER ENGINEERING  
DEPARTMENT**

**CENG 408**

**Implementation**

**TARAS: Agricultural Analysis System**

ARDA KAHRAMAN - 202111002  
HATİCE ELİF ARABACI - 202211008  
ÖYKÜ EYLÜL BOZDAĞ - 202311410  
MUSTAFA BERK KESKİN - 202211042

# TABLE OF CONTENTS

|                                             |           |
|---------------------------------------------|-----------|
| <b>1. Implementation.....</b>               | <b>3</b>  |
| <b>1.1. System Overview.....</b>            | <b>3</b>  |
| <b>2. Hardware Implementation.....</b>      | <b>4</b>  |
| <b>2.1. Sensor Node Design.....</b>         | <b>4</b>  |
| 2.2. Sensor Data Acquisition.....           | 5         |
| 2.3. Communication Layer.....               | 5         |
| 2.4. Power Management.....                  | 6         |
| <b>3. Backend &amp; Data Pipeline.....</b>  | <b>7</b>  |
| <b>3.1. Database Design.....</b>            | <b>7</b>  |
| 3.2. Data Processing Pipeline.....          | 7         |
| 3.3. Irrigation Recommendation Engine.....  | 8         |
| <b>4. Computer Vision Integration.....</b>  | <b>9</b>  |
| <b>5. LLM-Based Advisory System.....</b>    | <b>10</b> |
| <b>6. Carbon Footprint Calculation.....</b> | <b>10</b> |
| <b>7. Mobile Application.....</b>           | <b>11</b> |
| <b>8. System Integration.....</b>           | <b>12</b> |

# 1. Implementation

This chapter describes the engineering implementation of the TARAS platform, covering the hardware sensing infrastructure, communication protocols, cloud backend services, ML integration, and mobile application. The focus is on how each subsystem was built, connected, and deployed as a unified precision agriculture system.

## 1.1. System Overview

TARAS is an intelligent agriculture platform that integrates IoT-based sensing, digital-twin visualization, AI-driven irrigation recommendation and computer-vision-based plant disease detection into a single, cohesive system. The platform is designed to help farmers optimize water usage, detect crop diseases early and make data-driven decisions through an accessible mobile interface.

The system comprises six core subsystems:

**IoT Sensor Nodes:** Battery-powered ESP32-C6 microcontrollers equipped with capacitive soil-moisture sensors and SHT31 temperature/humidity sensors, deployed across the field to collect environmental data at regular intervals.

**Gateway:** A mains-powered ESP32 WROOM-32 device that aggregates sensor data from up to 20 nodes via ESP-NOW and forwards it to the cloud backend over Wi-Fi using HTTP POST and [Socket.IO](#).

**Cloud Backend:** A Node.js/Express API server deployed on AWS EC2, backed by PostgreSQL, providing RESTful endpoints for telemetry ingestion, irrigation analysis, disease detection, LLM-based advisory, and real-time data streaming via [Socket.IO](#).

**Machine Learning & Computer Vision:** A three-model deep-learning ensemble (one ConvNeXt-V2 and two Swin Transformers) for 14-class plant disease classification, served via ONNX Runtime on AWS Lambda, complemented by an on-device distilled EfficientNet-B0 (TFLite) for offline live scanning, alongside a rule-based irrigation recommendation engine running on the backend.

**Irrigation Recommendation Module:** A decision-support module that uses real-time soil moisture, temperature, humidity, and crop-specific thresholds to determine irrigation needs, recommend water amount or timing, and provide a reason for each suggestion. The module runs within the Node.js/Express backend, sharing the same compute and PostgreSQL database as the rest of the API.

**Mobile Application:** A React Native (with Expo) cross-platform application providing a digital-twin visualization, disease detection analysis, irrigation recommendations, an advisory AI chatbot, and carbon footprint estimation.

The end-to-end data flow operates as follows: sensor nodes collect soil moisture, temperature, and humidity readings at configurable intervals (default: every 5 minutes). Readings are buffered in RTC memory and transmitted in batches via ESP-NOW to the gateway. The gateway validates, formats, and POSTs the telemetry to the cloud API, where it is persisted in PostgreSQL and made available to the irrigation engine, digital twin, and mobile clients in real time via [Socket.IO](#).

## 2. Hardware Implementation

### 2.1. Sensor Node Design

Each sensor node is built around the ESP32-C6 SuperMini module, a RISC-V-based microcontroller featuring integrated 802.11b/g/n/ax Wi-Fi and Bluetooth 5 LE. The C6 variant was selected for its superior power efficiency compared to the standard ESP32, enabling battery-powered operation measured in months rather than days. The CPU base clock is set to 40 MHz from the crystal oscillator (bypassing the PLL), which is sufficient for I<sup>2</sup>C communication at 100 kHz and ADC sampling. When ESP-NOW transmission is required, the clock is dynamically raised to 80 MHz (the minimum for WiFi radio operations) and returned to 40 MHz afterward, minimizing average power consumption.

The sensor node interfaces with two primary sensing elements:

**SHT31 Temperature/Humidity Sensor:** Connected via I<sup>2</sup>C (SDA: GPIO 4, SCL: GPIO 7, address 0x44). This digital sensor provides high-accuracy temperature ( $\pm 0.3^{\circ}\text{C}$ ) and humidity ( $\pm 2\%$  RH) measurements. No soft reset is performed at boot; the SHT31's single-shot measurement mode operates correctly from any prior state, and omitting the reset saves approximately 40 ms per wake cycle.

**Capacitive Soil Moisture Sensor:** Connected to the ADC on GPIO 0. Unlike resistive sensors, capacitive probes do not corrode over time, making them suitable for long-term field deployment. Raw ADC readings are scaled and converted to soil moisture percentage using per-node calibration parameters (minimum and maximum raw soil moisture readings) stored in the backend.

The pin configuration of the ESP32-C6 SuperMini sensor node is summarized below:

| GPIO | Function               | Notes                      |
|------|------------------------|----------------------------|
| 4    | SDA (I <sup>2</sup> C) | SHT31 temperature/humidity |
| 7    | SCL (I <sup>2</sup> C) | SHT31 temperature/humidity |
| 0    | ADC                    | Capacitive soil moisture   |

## 2.2. Sensor Data Acquisition

Sensor readings are acquired at a configurable sampling interval, defaulting to 300 seconds (5 minutes) in production. At each wake cycle, the node initializes the I<sup>2</sup>C bus, reads the SHT31 sensor for temperature and humidity, samples the ADC for soil moisture, and stores the result as a compact 4-byte *stored\_reading\_t* structure in RTC memory. Temperature is stored as an int16 multiplied by 100 (preserving two decimal places), humidity as a uint8 (the raw value divided by 50 for storage, then multiplied by 50 on decode, yielding  $\pm 0.5\%$  RH precision), and soil moisture as a uint8 multiplied by 16. Sensor failure is indicated by sentinel values (0xFF for humidity and soil, -30000 for temperature).

The RTC buffer can hold up to 2,000 readings, equivalent to approximately 7 days of data at a 5-minute interval. When the buffer reaches the configured send threshold (based on minimum buffer cycles, maximum buffer cycles, or significant threshold changes in temperature, humidity, or soil moisture), readings are expanded to 7-byte *reading\_entry\_t* structures and transmitted to the gateway.

Calibration of soil moisture values is performed at the backend level. Each sensor node has its minimum and maximum raw ADC readings stored in the database. When a reading is received, the backend maps the raw value to a 0–100% moisture scale using linear interpolation between these calibration bounds.

## 2.3. Communication Layer

The communication architecture uses a two-hop design: sensor nodes communicate with the gateway via ESP-NOW (a connectionless, low-overhead protocol operating on the 2.4 GHz Wi-Fi band), and the gateway forwards data to the cloud over standard Wi-Fi.

**ESP-NOW Protocol:** All ESP-NOW packets are authenticated using HMAC-SHA256 signatures with per-node encryption keys established during the pairing process. The protocol supports multiple packet types including PAIR\_BEACON (gateway discovery), PAIR\_ACCEPT/PAIR\_COMPLETE (credential exchange), SENSOR\_DATA (batched readings), DATA\_ACK (acknowledgments), and CONFIG\_UPDATE (remote configuration changes). Each SENSOR\_DATA packet can carry up to 33 readings (limited by the 250-byte ESP-NOW maximum payload). Both devices are forced to 802.11b/g/n mode to ensure cross-compatibility between the ESP32-C6 (which supports WiFi 6/802.11ax) and the ESP32 WROOM-32 gateway. To prevent replay attacks, the gateway rejects duplicate HMAC counters from each node (duplicate-only rejection rather than strict monotonic ordering, to tolerate counter resets after deep-sleep anomalies). Each sensor node persists its HMAC counter to NVS

flash every 10 increments, ensuring counter continuity across reboots without excessive flash wear. For development and factory provisioning, the gateway also supports serial-based credential input as a fallback to BLE (activated by sending 'SERIAL' during the BLE advertising window).

**Gateway Forwarding:** The gateway is a mains-powered ESP32 WROOM-32 running at 80 MHz (sufficient since ESP-NOW is interrupt-driven and HTTP POST is WiFi-bound). Upon receiving and HMAC-verifying a sensor data packet, it sends a signed ACK back to the sensor node and constructs a JSON payload containing the readings. This payload is transmitted to the backend via HTTP POST to the `/api/sensors/device/data` endpoint, authenticated with a device-key header (X-Device-Key). If the HTTP transmission fails (e.g., due to connectivity loss), the gateway buffers the data in a compact format on SPIFFS (up to 64 KB, approximately 8,877 readings or 6.2 days of offline storage for 5 nodes). Buffered records are flushed every 30 seconds when connectivity is restored. To minimize sensor node radio-on time, the gateway employs a deferred ACK pipeline: it sends a signed acknowledgment to the sensor node immediately upon frame validation, then queues the data internally and batches the HTTP POST to the backend after all pending packets from that wake cycle have been processed.

**Real-Time Streaming:** In addition to HTTP telemetry, the gateway maintains a persistent Socket.IO connection to the backend for real-time events including gateway authentication, heartbeat signals, node discovery notifications, and pairing completion events.

**Diagnostics:** The protocol defines a PKT\_DIAGNOSTIC (0x12) packet type that sensor nodes transmit periodically alongside data packets. Diagnostic payloads use an enum-based, variable-length format covering 10 failure categories: send failures, ACK misses, sensor read failures, soil probe failures, NVS errors, WiFi failures, I<sup>2</sup>C bus errors, channel recovery events, buffer overflows, and boot/reset reason codes. The gateway forwards these to the backend, where they are stored in the `sensor_diagnostics` table for fleet health monitoring.

## 2.4. Power Management

Battery life is a critical design parameter for field-deployed sensor nodes. The system employs several power optimization strategies:

**Deep Sleep:** In production mode, the sensor node enters deep sleep between sampling cycles, drawing microamp-level current. The wake cycle (sensor read, optional data transmission) takes approximately 34 ms for non-sending cycles and 195 ms for sending cycles, resulting in a duty cycle of approximately 0.01%.

**TX Power:** Transmission power is set to 20 dBm (API value 78), the maximum for the ESP32-C6, to ensure reliable through-wall range across the field. While higher TX power increases instantaneous current draw during the brief transmission window, the deep-sleep duty cycle ensures that radio-on time represents a negligible fraction of total energy consumption.

**Dynamic CPU Frequency:** The base clock is set to 40 MHz from the crystal oscillator (no PLL), approximately 60% below the default 160 MHz. The CPU dynamically switches to 80 MHz only during WiFi/ESP-NOW radio operations and returns to 40 MHz immediately afterward, keeping average power consumption well below what a fixed 80 MHz clock would draw.

With these optimizations, a sensor node powered by a 1000 mAh Li-Ion battery can operate for months in normal production mode at a 5-minute sampling interval.

## 3. Backend & Data Pipeline

The TARAS backend is implemented as a TypeScript/Express server running on AWS EC2, with PostgreSQL hosted on AWS RDS as the primary data store. The server exposes RESTful API endpoints and maintains WebSocket connections via Socket.IO for real-time data delivery to mobile clients.

### 3.1. Database Design

The data model is managed through Prisma ORM with a hierarchical structure: **User** → **Farm** → **Field** → **Zone** → **SensorNode** → **SensorReading**. Each Zone contains a ZoneDetail record holding irrigation parameters such as crop coefficient (Kc), irrigation gain (mm/min), and target soil moisture values. The IrrigationJob model tracks recommendation history with statuses including PENDING, EXECUTED, SKIPPED, and ANALYZED.

Key entities in the data model include: sensor\_nodes (device registration, calibration bounds, MAC address, status), sensor\_readings (timestamped telemetry with soil moisture, temperature, humidity, and error bitmask), irrigation\_jobs (recommendation output, predicted vs. actual soil moisture for calibration), disease\_results (image URL, classification label, confidence), and chat\_sessions/chat\_messages (LLM advisory conversation history).

### 3.2. Data Processing Pipeline

Incoming telemetry follows a structured processing pipeline. When the backend receives a POST from the gateway, it validates the device key against registered sensor nodes, checks the data schema (rejecting records with missing or out-of-range values), maps raw ADC soil moisture readings to calibrated percentage values using the per-node min/max bounds, and persists the validated readings as SensorReading records via Prisma's createMany batch insert. The system also inspects the sensor error bitmask in each reading and creates rate-limited Alert records (with severity levels INFO, WARNING, or CRITICAL) when sensor faults are detected.

After successful persistence, the backend emits a sensor-update event via Socket.IO to all clients subscribed to the relevant field room, enabling real-time dashboard updates in the mobile application.

### 3.3. Irrigation Recommendation Engine

The irrigation recommendation system is designed as a rule-based decision model combined with a physically inspired water-demand estimation and an adaptive calibration layer. This architecture was chosen over a ML approach to ensure interpretability, controllability, and stable behavior under real-world constraints where training data is limited.

The engine operates with a clear separation of responsibilities: fixed thresholds determine when to irrigate,  $ET_0$ -based estimation determines how much water is needed, and a calibration layer corrects how much water is actually applied.

#### **Decision Logic (When to Irrigate)**

The decision of whether irrigation is required is determined using fixed soil-moisture thresholds defined per crop growth stage (seedling, vegetative, flowering/fruiting, ripening). Each stage specifies a recommended minimum/maximum moisture range, critical minimum/maximum bounds, and a target soil moisture value. For example, the flowering and fruiting stages require higher minimum moisture (65%) reflecting the crop's increased water sensitivity, while the vegetative stage tolerates a wider range (50–65%). These thresholds are intentionally kept fixed as design parameters to maintain system stability and interpretability.

#### **Water Demand Estimation (How Much to Irrigate)**

Once irrigation is triggered, the required water amount is estimated using a simplified FAO-inspired approach. Crop evapotranspiration is computed as  $ET_c = ET_0 \times K_c$ , where  $ET_0$  is the reference evapotranspiration and  $K_c$  is a stage-dependent crop coefficient (e.g., 0.60 for seedling, 1.15 for flowering/fruiting). The total required irrigation combines the evapotranspiration-based demand with a soil-moisture deficit adjustment: the deficit ( $target\_sm - current\_sm$ ) is multiplied by an initial heuristic coefficient of 0.5 to convert percentage-point deficit to mm of estimated water demand. If the irrigation gain (mm/min) is available for the field, duration is computed directly; otherwise, the system falls back to base duration estimates per irrigation mode.

#### **Calibration Layer**

The calibration layer is applied only to irrigation amount and duration, not to thresholds. It compensates for sensor inaccuracies, soil variability, irrigation system efficiency, and environmental differences. The calibration factor is computed from the running average of prediction errors: after each recommendation, the system compares the predicted soil moisture one hour

post-irrigation with the actual observed value. Once at least 10 recommendations have been executed, the calibration factor is computed as  $1 + (\text{average\_prediction\_error} / 100)$ , increasing water application when the system consistently underpredicts consumption and decreasing it otherwise. Below 10 recommendations, all calibration factors default to 1.0.

### **Urgency and Timing**

The engine assigns urgency levels based on proximity to critical thresholds: HIGH when soil moisture is within 5 points of the critical minimum, MEDIUM when within 5 points of the recommended minimum, and LOW otherwise. High-urgency recommendations trigger immediate irrigation; medium-urgency recommendations schedule irrigation within preferred agricultural time windows (05:00–09:00 or 18:00–23:00); low-urgency conditions produce no recommendation. ETc-based atmospheric demand can elevate urgency from medium to high when current evapotranspiration exceeds 1.2 times the rolling 7-day mean.

## **4. Computer Vision Integration**

The disease detection module is deployed as a containerized AWS Lambda function, separate from the main backend to enable independent scaling and avoid impacting API latency. The inference pipeline operates as follows:

When a user uploads a leaf image through the mobile application, the image is sent to the backend API. The backend stores the original image in AWS S3, then invokes the Lambda function with an S3 reference payload (`{s3_bucket, s3_key}`), avoiding the 6 MB Lambda payload size limit. The Lambda function loads the three-model ONNX ensemble (one ConvNeXt-V2 and two Swin Transformers) via ONNX Runtime, preprocesses the image (resize to 224×224 or 384×384 pixels), runs inference, and returns a classification result containing the predicted disease label, confidence score, and probability distribution across all 14 classes.

The 14 target classes span tomato, corn, and other crops: Bacterial Spot, Corn Common Rust, Corn Gray Leaf Spot, Corn Northern Leaf Blight, Early Blight, Late Blight, Leaf Mold, Mosaic Virus, Powdery Mildew, Septoria Leaf Spot, Spider Mites, Target Spot, Yellow Leaf Curl Virus, and Healthy.

The Lambda function runs on arm64 (AWS Graviton2) with 1,769 MB of memory and a 180-second timeout. Cold starts take approximately 14 seconds (loading the three ensemble models), while warm invocations complete in approximately 6 seconds. The backend stores the classification result, confidence, and image URL in the `disease_results` table and returns the result to the mobile client. The Lambda function is invoked synchronously (InvocationType: RequestResponse) but called in a non-blocking pattern from the API handler, so the client polls for completion rather than waiting on the HTTP request.

## 5. LLM-Based Advisory System

The TARAS advisory module integrates Large Language Models through standard REST APIs, avoiding dependency on vendor-specific SDKs. The system adopts a hybrid architecture that combines agentic AI with a Retrieval-Augmented Generation (RAG) pipeline to ensure accurate and context-aware recommendations.

When a farmer submits a query, the AI agent orchestrates multiple tools to gather relevant information. Real-time zone telemetry, such as soil moisture levels and critical thresholds, is retrieved from the PostgreSQL database via dedicated data access tools. In parallel, a RAG pipeline retrieves agronomic knowledge (e.g., irrigation strategies, crop-specific guidelines) from a curated knowledge base using PostgreSQL full-text search.

The retrieved documents are then used by the model during response generation, ensuring that outputs are grounded both in live field data and validated domain knowledge. This significantly reduces hallucinations while improving the reliability of recommendations.

To handle provider-specific API requirements, a dedicated adapter service formats requests into the exact JSON schema expected by the selected AI endpoint. This decoupled design allows the system to switch or upgrade the underlying model without modifying the core backend, agent logic, or retrieval pipeline.

## 6. Carbon Footprint Calculation

The carbon footprint module is implemented as a backend-driven calculation service that estimates farm-level operational emissions based on user-provided activity data. The system considers three primary activity categories: electricity consumption, fuel usage (diesel, LPG, gasoline), and nitrogen fertilizer application.

The calculation follows a factor-based approach defined as:

$$\text{Carbon Emissions} = \text{Activity Amount} \times \text{Emission Factor}$$

Emission factors are predefined in the system based on standardized sources such as IPCC and FAO. Each activity type is mapped to a fixed coefficient, ensuring consistency and transparency in the calculation process.

The module is built on a log-based data structure where each user input is stored as an activity record. Each `carbon_log` entry includes the associated field, activity type, activity date, activity amount, and the computed emission value. Activity types are defined separately to standardize categories such as electricity, fuel, and fertilizer, while emission factors are stored independently and linked to activity types with validity ranges, allowing future updates without affecting historical calculations.

During computation, the emission value is calculated at the moment each activity is recorded. When a new activity log is created, the system applies the corresponding emission factor to the activity amount and stores the result as `emission_amount` within the `carbon_log` record. In addition, the system updates the cumulative carbon value stored at the farm level by incrementing the `total_CO2e` field in the farm table. This value represents the total emissions generated by the farm over time and is maintained independently from filtered or date-based queries used in the user interface.

When filters such as date range or activity type are applied, the system does not modify stored values. Instead, it performs a read-time aggregation by querying the relevant `carbon_log` entries that match the selected filters. The filtered logs are retrieved from the database, and their stored `emission_amount` values are summed to generate the filtered result. This allows the user to view emissions for a specific period, activity type, or tag without affecting the cumulative `total_CO2` value maintained at the farm level.

## 7. Mobile Application

The TARAS mobile application is built with React Native using the Expo framework, supporting both Android and iOS from a single codebase. The application communicates with the backend via RESTful APIs (authenticated with JWT tokens stored in `AsyncStorage`) and receives real-time sensor updates through Socket.IO WebSocket connections.

The primary screens and features include:

**Digital Twin Dashboard:** The central screen renders an interactive 3D visualization of the agricultural field using React-Three/Fiber and custom GLSL shaders. Real-time sensor data (soil moisture and temperature) is interpolated across field zones and displayed as a color-coded heatmap. Users can rotate the perspective by dragging and tap specific zones to view detailed readings. Zones where values fall outside optimal ranges are visually highlighted to alert the farmer.

**Disease Detection:** Users can capture a leaf image using the device camera or upload from the gallery. The image is sent to the backend, which invokes the Lambda-based ensemble for classification. The result (disease type, confidence) is displayed on screen along with treatment recommendations generated by the system.

**Irrigation Recommendations:** The timetable screen presents a schedule of irrigation tasks based on the rule-based engine's output, showing recommended timing, duration, urgency level and reasoning.

**AI Advisory Chatbot:** A floating button accessible from all primary screens opens the conversational AI interface, where farmers can ask questions about their field conditions and receive context-aware guidance from the LLM.

**Carbon Footprint Calculator:** Farmers input energy, fuel, and fertilizer usage data through a structured form. The backend computes estimated CO<sub>2</sub>e emissions using predefined emission factors and returns a breakdown for visualization.

The application supports light/dark/system theme modes, Turkish/English localization (all UI strings are managed through a centralized strings.ts file), and multi-user role-based access (farmer and stakeholder roles with different visibility permissions).

## 8. System Integration

All subsystems are integrated through the cloud backend, which serves as the central hub for data flow, service orchestration, and client communication. The end-to-end pipeline operates as follows:

Sensor data is collected by ESP32-C6 nodes, transmitted via ESP-NOW to the gateway, forwarded via HTTP POST to the backend API, validated and persisted in PostgreSQL, and streamed to mobile clients via Socket.IO. The irrigation engine periodically retrieves the latest sensor time series, evaluates rule-based thresholds, computes ET<sub>c</sub>-based water demand, applies calibration corrections, and stores the recommendation as an IrrigationJob. Disease detection operates as an event-driven pipeline: image upload triggers Lambda invocation, inference results are stored and returned to the client. The LLM advisory system pulls the latest sensor data, irrigation recommendations, and disease results to construct contextual prompts. The digital twin visualization on the mobile app consumes real-time sensor streams and renders interpolated heatmaps over the field geometry.